

Career Platform

Step-by-step deployment guide for beginners

Frontend (Next.js) and backend (FastAPI) on one AWS EC2 server, with PostgreSQL, and public access through Cloudflare Tunnel.

Domain	career.qualcsi.com
Repository	github.com/qulacsi-Tech/career-exploration-lead
Server	AWS EC2, Ubuntu 24.04 LTS, t3.medium, region ap-south-1 (Mumbai)
Purpose	Testing and demonstration. Read section 2 before inviting anyone.

Contents

1. How the system fits together
2. Before you start: what you need, and what it costs
3. Part A: Create the AWS account items (key pair, security group, server, IP address)
4. Part B: Create the Cloudflare Tunnel (public address with HTTPS)
5. Part C: Connect to the server
6. Part D: Prepare the server (updates, swap, firewall, Docker)
7. Part E: Download the code
8. Part F: Write the configuration and secrets
9. Part G: Start the database and the backend
10. Part H: Create the tables and sample data (once only)
11. Part I: Start the proxy and the tunnel
12. Part J: Build and start the frontend
13. Part K: Test the site
14. Part L: Close the extra ports and set up backups
15. Part M: Everyday operations (update, roll back, stop, restart)
16. Part N: When something goes wrong
17. Part O: Security and demo rules
18. Appendix: Words used in this guide

1. How the system fits together

There are four running parts on one server. You only need to know what each one does.

Part	What it is	Who reaches it
Cloudflare Tunnel (cloudflared)	A connector on the server that makes an outgoing link to Cloudflare. Visitors' requests come down this link.	Only Cloudflare, over the outgoing link
Caddy	A small web router. It sends <code>/api/...</code> requests to the backend and everything else to the frontend.	Only the tunnel, inside the server
Frontend (Next.js)	The public website and the admin panel.	Only Caddy, inside the server
Backend (FastAPI) and database (PostgreSQL)	The API, and the data store.	Only Caddy and the backend, inside the server. The database is never exposed.

Visitors type `https://career.qualcsi.com`. Cloudflare provides the HTTPS padlock. The request travels through the tunnel to Caddy, then to the frontend or backend. The server has no public web ports open.

Why a tunnel? Your server stays closed to the internet except for SSH (for you). You do not have to manage HTTPS certificates. It is also easy to switch off: stop the tunnel and the site disappears from the internet at once.

2. Before you start

2.1 What you need

- An AWS account with billing enabled. You need permission to create EC2 instances and Elastic IPs.
- A free Cloudflare account, and control of the domain **qualcsi.com** (its nameservers or DNS).
- Read access to the GitHub repository `qualacsi-Tech/career-exploration-lead`.
- A computer with Windows, macOS or Linux, and an SSH tool. Windows 10 and 11 include `ssh` in PowerShell.
- A password manager, for the secrets you will create in Part F.

2.2 Cost

Item	Approximate monthly cost (USD)
EC2 t3.medium, running all the time	about 30
Storage, 30 GB	about 3
Elastic IP (while attached to a running instance it is free; it is billed when the instance is stopped)	about 0 to 4
Cloudflare Tunnel and DNS (free plan)	0

Stop the instance when nobody is testing, to save money. Section M explains how.

2.3 Decisions already made

Decision	Value used in this guide
Region	ap-south-1 (Mumbai)
Instance	t3.medium, Ubuntu 24.04 LTS
Database	PostgreSQL 16, inside the server, in a container
Public access	Cloudflare Tunnel (no open web ports)

Important: this is a test server. The sample content is not real data. Do not send real student information through it. Read section O before sharing the address with anyone.

Part A. Create the AWS items

Use the AWS website: **console.aws.amazon.com**. Sign in, then follow each step in order.

A1. Choose the region

Step 1.

In the top-right corner of the AWS console, click the region name. Choose **Asia Pacific (Mumbai) ap-south-1**.

Every item in this part must be created in the same region. If you change region later, the items will not be visible.

A2. Create a key pair (your login key for the server)

Step 2.

Search for **EC2** at the top, and open it. In the left menu, click **Key pairs**, then **Create key pair**.

- Name: `career-demo`
- Key pair type: **ED25519**
- Private key file format: **.pem**

Click **Create key pair**. Your browser downloads `career-demo.pem`.

Keep this file private. Anyone with it can log in to your server. Do not email it or upload it anywhere. On Windows, move it to a folder such as `C:\Users\<you>\.ssh\`.

A3. Create a security group (the server's firewall rules)

Step 3.

In EC2, click **Security Groups** in the left menu, then **Create security group**.

- Security group name: `career-web`
- Description: `Demo server: SSH only`
- VPC: leave the default
- Inbound rules: click **Add rule**
 - Type: **SSH**, Port: 22, Source type: **My IP**
- Outbound rules: leave the default (all traffic)

Click **Create security group**.

Only SSH is open to the internet, and only from your current address. Visitors reach the site through Cloudflare, so no web ports are needed. If your internet address changes, edit this rule (Security Groups → career-web → Inbound rules → Edit) and choose **My IP** again.

A4. Launch the server (EC2 instance)

Step 4.

In EC2, click **Instances**, then **Launch instances**.

Setting	What to choose
Name and tags: Name	career-demo
Application and OS Images (AMI)	Ubuntu , then Ubuntu Server 24.04 LTS (HVM), SSD Volume Type, 64-bit (x86)
Instance type	t3.medium (2 vCPU, 4 GiB memory)
Key pair (login)	career-demo
Network settings: Firewall (security groups)	Select existing security group , then career-web
Configure storage	30 GiB, gp3

Click **Launch instance**. Click **View all instances**. Wait until **Instance state** shows **Running** and **Status check** shows **2/2 checks passed** (about 2 minutes).

A5. Give the server a fixed address (Elastic IP)

Step 5.

Without this, the server's address changes every time it restarts.

1. In the left menu, click **Elastic IPs**, then **Allocate Elastic IP address**, then **Allocate**.
2. Select the new address in the list. Click **Actions**, then **Associate Elastic IP address**.
3. Resource type: **Instance**. Choose career-demo . Click **Associate**.
4. Write the address down. This guide calls it **EIP**. An example is 13.201.10.25 .

Check A: In **Instances**, career-demo shows **Running** and its **Public IPv4 address** equals your EIP.

Part B. Create the Cloudflare Tunnel

This gives the site its public address and its HTTPS padlock. You do this in the Cloudflare website, **one.dash.cloudflare.com**.

B1. Check the domain is on Cloudflare

Step 6.

In Cloudflare, open **Domains** (or **Websites**) and check that **qualcsi.com** is listed as **Active**.

- If it is **not** listed: click **Add a domain**, enter `qualcsi.com`, and follow the screens. Cloudflare gives you two nameservers. Set them at the company where you bought the domain, in its DNS or nameserver settings. It can take up to a day, usually less.
- If you cannot move the domain, use the note in step 9 instead.

B2. Create the tunnel

Step 7.

1. Go to **one.dash.cloudflare.com** and choose your account. If asked, choose the free plan.
2. In the left menu, open **Networks**, then **Connectors**, then **Cloudflare Tunnels**.
3. Click **Create a tunnel**. Choose **Cloudflared**, then **Next**.
4. Tunnel name: `career-demo`. Click **Save tunnel**.
5. On the install screen, choose the **Docker** tab. You will see a command that starts with `docker run`.
6. **Copy only the token**. It is the long text that starts with `eyJ`, after the word `--token`. Do not run the command. The server already runs the connector. Paste the token into a secure note now.

The token lets anyone run your tunnel. Treat it like a password. Do not put it in a document, email or chat.

B3. Add the public address

Step 8.

On the same tunnel page, open the **Public hostname** tab, then click **Add a public hostname**.

Field	Value
Subdomain	<code>career</code>
Domain	<code>qualcsi.com</code>
Path	leave empty
Service type	HTTP
URL	<code>caddy:80</code>

Click **Save hostname**. Cloudflare creates the DNS record for `career.qualcsi.com` by itself.

Step 9. (Only if step 6 was not possible.)

In the DNS settings of **qualcsi.com**, add a **CNAME** record: name `career`, target `<tunnel-id>.cfargotunnel.com`. The tunnel ID is shown on the tunnel page. Turn the proxy (orange cloud) on.

Check B: The tunnel page shows the tunnel as **Inactive** or **Down**. That is correct until the server connects in Part I.

Part C. Connect to the server

Step 10.

On your computer, open **PowerShell** (Windows) or **Terminal** (macOS and Linux). Run the command below. Replace the two placeholders.

```
ssh -i C:\Users\<you>\.ssh\career-demo.pem ubuntu@EIP
```

The first time, you will see a question about the server's fingerprint. Type `yes` and press Enter.

When the prompt changes to `ubuntu@ip-...:~$`, you are on the server. Every command from here on runs on the server, unless the step says otherwise.

Check C: Type `whoami`. It prints `ubuntu`.

Mistakes to avoid. Use `Ctrl+Shift+V` or right-click to paste into the terminal. Copy each command exactly, including any capital letters. If a command shows a red error, read it before running the next one.

Part D. Prepare the server

D1. Update Ubuntu and install tools

Step 11.

```
sudo apt update && sudo apt -y full-upgrade
sudo apt -y install ca-certificates curl git ufw fail2ban unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

In the last step, a blue screen asks about automatic updates. Choose **Yes**. Press Enter.

D2. Add swap memory

The website build needs more memory than the server has, so we add 4 GB of swap (disk used as extra memory).

Step 12.

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

Check D2: The line starting with `Swap:` shows `4.0Gi`.

D3. Firewall: allow SSH only

Step 13.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw --force enable
sudo ufw status verbose
```

Check D3: The status lists **OpenSSH** and nothing else allowed.

D4. Install Docker

Docker runs each part of the platform in its own sealed container, so setup is repeatable.

Step 14.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable"
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt -y install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
sudo usermod -aG docker ubuntu
```

Step 15.

Leave the server and connect again, so the new group membership applies:

```
exit
```

Reconnect with the same command as step 10. Then:

```
docker version
docker compose version
docker run --rm hello-world
```

Check D4: hello-world prints *Hello from Docker!*

Part E. Download the code

The repository is private. The server gets its own read-only key to download it.

Step 16. Create the server's key.

```
ssh-keygen -t ed25519 -C "career-demo-server" -f ~/.ssh/github_deploy -N ""  
cat ~/.ssh/github_deploy.pub
```

The last command prints one long line that starts with `ssh-ed25519`. Select it and copy it.

Step 17. Add the key to GitHub (on your computer, in a web browser).

1. Open `github.com/qulacsi-Tech/career-exploration-lead`.
2. Click **Settings** (top menu of the repository), then **Deploy keys** in the left menu.
3. Click **Add deploy key**. Title: `career-demo-server`. Paste the key. Leave **Allow write access** unticked.
4. Click **Add key**. You may be asked for your GitHub password.

Step 18. Clone the repository (back on the server).

```
cat >> ~/.ssh/config <<'EOF'  
Host github.com  
  IdentityFile ~/.ssh/github_deploy  
  IdentitiesOnly yes  
EOF  
chmod 600 ~/.ssh/config  
ssh-keyscan github.com >> ~/.ssh/known_hosts  
sudo mkdir -p /opt/career-platform  
sudo chown ubuntu:ubuntu /opt/career-platform  
git clone git@github.com:qulacsi-Tech/career-exploration-lead.git /opt/career-platform  
cd /opt/career-platform  
git log --oneline -1
```

Check E: The last line shows a commit (short code and a message). The code is on the server at `/opt/career-platform`.

Part F. Write the configuration and secrets

What is a secret? A password or key the system uses. Each one is random and long, and it is written only on the server. Never paste a secret into a chat, a document, a screenshot or a Git commit.

Step 19. Create three secret values.

Run the command three times. Each run prints a different random value. Copy each into your password manager, with a label.

```
openssl rand -hex 24      # label: POSTGRES_PASSWORD
openssl rand -hex 16      # label: ADMIN_PASSWORD (for the demo admin login)
openssl rand -hex 32      # label: SECRET_KEY
```

Step 20. Create the environment file.

```
cd /opt/career-platform/deploy
cp .env.example .env
nano .env
```

The `nano` editor opens. Use the arrow keys to move. Change these lines (keep the rest as it is):

```
DOMAIN=career.qualcsi.com
CLOUDFLARE_TUNNEL_TOKEN=eyJ...the token from step 7...
POSTGRES_PASSWORD=...first secret from step 19...
ADMIN_EMAIL=admin@career.qualcsi.com
ADMIN_PASSWORD=...second secret from step 19...
ADMIN_NAME="Demo Admin"
```

To save and close nano: press `Ctrl+O`, then `Enter`, then `Ctrl+X`.

Step 21. Create the backend file.

```
cp backend.env.example backend.env
nano backend.env
```

Change these two lines:

```
DATABASE_URL=postgresql+asyncpg://postgres:...first secret from step 19...@db:5432/leadgendb
SECRET_KEY=...third secret from step 19...
```

The password in `DATABASE_URL` must be exactly the same as `POSTGRES_PASSWORD`. If they differ, the backend cannot connect to the database.

Save and close nano, as before.

Step 22. Lock both files so only you can read them.

```
chmod 600 .env backend.env  
ls -l .env backend.env
```

Check F: Both lines start with `-rw-----`.

Part G. Start the database and the backend

Step 23. Start the first three containers.

```
cd /opt/career-platform/deploy
docker compose -f docker-compose.prod.yml --env-file .env up -d --build db backend
docker compose -f docker-compose.prod.yml ps
```

The first build takes 2 to 5 minutes. Wait until the list shows `db` with `(healthy)`. If it is still `starting`, wait 30 seconds and run the second command again.

Step 24. Check the backend answers.

```
docker compose -f docker-compose.prod.yml exec backend python -c "import
urllib.request;print(urllib.request.urlopen('http://localhost:8000/api/ping').read())"
```

Check G: The output is `b'{"status":"ok"}'`.

Part H. Create the tables and sample data

Run this part only once, on an empty database. Running the sample-data step twice creates duplicate colleges. If you need to start again, see section N.

Step 25. Create the database tables.

```
cd /opt/career-platform/deploy
set -a; . ./env; set +a
docker compose -f docker-compose.prod.yml --env-file .env run --rm backend alembic upgrade head
```

The output lists several lines ending in `Running upgrade ... -> 009`.

Step 26. Load the sample data and create the demo admin account.

```
docker compose -f docker-compose.prod.yml --env-file .env run --rm -e
ADMIN_EMAIL="$ADMIN_EMAIL" -e ADMIN_PASSWORD="$ADMIN_PASSWORD" -e ADMIN_NAME="$ADMIN_NAME"
backend python seed.py
```

Check H: The last line is `Seed complete`.

Part I. Start the proxy and the tunnel

Step 27. Start the router and the Cloudflare connector.

```
cd /opt/career-platform/deploy
docker compose -f docker-compose.prod.yml --env-file .env up -d caddy cloudflared
docker compose -f docker-compose.prod.yml logs --tail=30 cloudflared
```

Look for lines containing `Registered tunnel connection`. Four of them is normal.

Now check the tunnel dashboard: Cloudflare → Networks → Connectors → Cloudflare Tunnels → `career-demo`. Its status should change to **Healthy** within a minute. Press `Ctrl+C` to stop the log view if it keeps scrolling.

Step 28. Check the address works from the server.

```
curl -s https://career.qualcsi.com/api/ping
```

Check I: The output is `{"status": "ok"}`. If it is not, see section N, "Tunnel" problems.

Part J. Build and start the website

The website is built with the public address, so the tunnel must be working first (Part I).

Step 29. Build the frontend. This takes 5 to 15 minutes. Do not close the window.

```
cd /opt/career-platform/deploy
docker compose -f docker-compose.prod.yml --env-file .env build frontend
```

Step 30. Start it and check every container.

```
docker compose -f docker-compose.prod.yml --env-file .env up -d frontend
docker compose -f docker-compose.prod.yml ps
```

Check J: Every row (backend , caddy , cloudflared , db , frontend) shows up . The db row shows (healthy) .

Part K. Test the site

Do these checks from **your own computer**, not from the server. Use a private browser window for the admin checks.

#	Open this address	What you should see
1	<code>https://career.qualcsi.com</code>	The home page with the padlock.
2	<code>http://career.qualcsi.com</code>	It opens the HTTPS version (Cloudflare redirects).
3	<code>https://career.qualcsi.com/api/v1/home</code>	Text in curly brackets containing <code>"success": true</code> .
4	<code>https://career.qualcsi.com/courses</code>	A list of courses such as MBA and BBA.
5	<code>https://career.qualcsi.com/compare</code>	The comparison list.
6	<code>https://career.qualcsi.com/search?q=mba</code>	Matching colleges.
7	<code>https://career.qualcsi.com/sitemap.xml</code>	A page of XML addresses.
8	<code>https://career.qualcsi.com/exams/cat/practice</code>	Practice test cards. Open one, answer a question, press Submit, confirm: the result page appears.
9	<code>https://career.qualcsi.com/admin</code>	It opens the login page (this is correct).
10	Log in on <code>/login</code> with <code>admin@career.qualcsi.com</code> and the <code>ADMIN_PASSWORD</code> from step 19	The admin panel opens with its dashboard.
11	Admin → Content → Study Abroad. Change one word, press Save, open the public <code>/study-abroad</code> page, then change the word back.	The change shows on the public page. Your test confirms saving works.

Check K: All 11 checks pass. Then repeat checks 1, 4 and 8 on a phone using mobile data, which is a different network.

Part L. Close extra ports and set up backups

Only SSH should be open. The tunnel does not need any web ports on the server.

Step 31. Confirm the security group has no web rules.

In AWS: EC2 → Security Groups → `career-web` → Inbound rules. There must be exactly one rule: SSH, port 22, source **My IP**. If an HTTP or HTTPS rule exists, delete it.

Step 32. Set up a daily backup at 02:30.

```
chmod +x /opt/career-platform/deploy/backup.sh
(crontab -l 2>/dev/null; echo '30 2 * * * /opt/career-platform/deploy/backup.sh >> /opt/career-
platform/deploy/backups/backup.log 2>&1') | crontab -
/opt/career-platform/deploy/backup.sh
ls -lh /opt/career-platform/deploy/backups/
```

Check L: The list shows a file named `leadgendb-....sql.gz`.

Copy backups off the server regularly. In the AWS console, the simplest way is to download files with an SFTP program such as WinSCP, which connects with the same key. A better option, for later, is to copy them to a private Amazon S3 bucket.

Part M. Everyday operations

M1. Look at the logs

```
cd /opt/career-platform/deploy
docker compose -f docker-compose.prod.yml logs --tail=100 backend
docker compose -f docker-compose.prod.yml logs --tail=100 frontend
docker compose -f docker-compose.prod.yml logs --tail=100 cloudflared
```

Add `-f` after `logs` to follow live. Press `Ctrl+C` to stop.

M2. Restart one part

```
docker compose -f docker-compose.prod.yml restart frontend
```

M3. Update to a new version

```
cd /opt/career-platform
git pull
cd deploy
docker compose -f docker-compose.prod.yml --env-file .env run --rm backend alembic upgrade head
docker compose -f docker-compose.prod.yml --env-file .env up -d --build backend frontend
```

The database changes are applied first, then the new code starts.

M4. Roll back to the previous version

```
cd /opt/career-platform
git log --oneline -5
git checkout <previous-commit-code>
cd deploy
docker compose -f docker-compose.prod.yml --env-file .env up -d --build backend frontend
```

Replace `<previous-commit-code>` with the code from the list (for example `3cb39d1`). Database changes are not undone automatically. If a change went wrong, restore the database from a backup (section N).

M5. Stop the demo to save money

Stop the server's containers first, then the instance:

```
cd /opt/career-platform/deploy
docker compose -f docker-compose.prod.yml --env-file .env stop
```

In the AWS console: EC2 → Instances → select `career-demo` → Instance state → **Stop instance**. The Elastic IP stays attached, so the address is the same when you start again.

To start again: EC2 → Instances → **Start instance**. Then connect with SSH and run:

```
cd /opt/career-platform/deploy
docker compose -f docker-compose.prod.yml --env-file .env up -d
```

M6. Check disk space

```
df -h /
docker system df
docker image prune -f
```

`docker image prune` removes old, unused images. It is safe.

Part N. When something goes wrong

Find your symptom in the first column. Try the fix, then check again. If it still fails, copy the log output (section M1) and ask for help with the step number.

What you see	Likely cause	Fix
SSH: <i>Permission denied (publickey)</i>	Wrong key file, or wrong user name	Use the <code>.pem</code> from step 2 and the user <code>ubuntu</code> . On Windows, the key file must not be shared with other users (right-click → Properties → Security).
SSH: <i>Connection timed out</i>	Your internet address changed	EC2 → Security Groups → career-web → Edit inbound rule → source My IP again.
Tunnel shows Down; <code>cloudflared</code> keeps restarting	The token is wrong, or has spaces or line breaks	Copy the token again into <code>.env</code> (step 20), then run <code>up -d cloudflared</code> .
Cloudflare error 1033 or 502	A container behind the tunnel is down	<code>docker compose -f docker-compose.prod.yml ps</code> , then restart the one that is not Up.
<code>curl .../api/ping</code> does not work, but the containers are Up	The tunnel's public address points at the wrong service	Cloudflare → Tunnel → Public hostname → URL must be <code>caddy:80</code> .
Build of the frontend fails with <i>fetch failed</i>	The tunnel was not healthy during the build	Complete Part I first, then run step 29 again.
<code>backend</code> logs show a database error	<code>POSTGRES_PASSWORD</code> and the password in <code>backend.env</code> do not match	Make them identical (step 21), then <code>up -d backend</code> .
Admin login says <i>Could not sign in</i>	Wrong password, or the sample-data step (26) did not run	Use the ADMIN_PASSWORD from step 19. Check the output of step 26.
Site shows a blank page or an error after a change	A stale build or a failed update	Run M3 again. Check <code>logs --tail=100 frontend</code> .
Build runs out of memory	Swap not active	<code>free -h</code> ; if Swap is 0, repeat step 12.
Disk full	Old images or backups	Section M6; delete old files in <code>deploy/backups/</code> after copying them.

N1. Start over completely (deletes all data on the server)

```
cd /opt/career-platform/deploy
docker compose -f docker-compose.prod.yml --env-file .env down
docker volume rm career-platform_pgdata
set -a; . ./env; set +a
docker compose -f docker-compose.prod.yml --env-file .env run --rm backend alembic upgrade head
docker compose -f docker-compose.prod.yml --env-file .env run --rm -e
ADMIN_EMAIL="$ADMIN_EMAIL" -e ADMIN_PASSWORD="$ADMIN_PASSWORD" -e ADMIN_NAME="$ADMIN_NAME"
backend python seed.py
```

N2. Restore a backup

```
cd /opt/career-platform/deploy
ls -lt backups/ | head
gunzip -c backups/leadgendb-.sql.gz | docker compose -f docker-compose.prod.yml --env-file .env
exec -T db psql -U postgres -d leadgendb
```

Replace `<date>` with the file name from the list. Restoring over a live database can cause errors if tables already exist. Ask for help before restoring on a server that has data you care about.

Part O. Security and demo rules

Before you share the address with anyone:

- **Use sample data only.** The enquiry form (for counselling requests) stores what visitors type. Ask testers to use fake names and numbers, or switch the form off.
- **Keep the admin password private.** Anyone with it can change the site's content. Change it after the demo: log in, then Admin → Users.
- **Limit visitors if needed.** Cloudflare Access can ask testers for an email code before they see the site. Zero Trust → Access → Applications → Add a self-hosted application → domain `career.qualcsi.com`, then allow only the listed emails.
- **Keep the API documentation hidden if you do not want it public.** The page `/api/docs` lists every API operation. Ask for help to block it.
- **Never commit secrets.** The files `deploy/.env` and `deploy/backend.env` are excluded by the repository's ignore rules. Check with `git status` before you commit anything.
- **Keep the server updated.** Ubuntu installs security updates on its own (step 11). Update the platform with section M3 after each change.
- **Delete the demo when finished.** Stop the containers, stop the instance (section M5), then delete the instance, Elastic IP, and the Cloudflare tunnel, so no charges continue.

Known limits in this demo

- One server. If it stops, the site stops until it is started again.
- Visitors do not have student accounts yet. Only the admin panel has a login.
- Five collections point at ranking lists that do not exist in the database. Their colleges are shown in editor order, not ranking order.
- Search uses the database, not Meilisearch.

Appendix: Words used in this guide

Word	Meaning
AWS / EC2	Amazon Web Services. EC2 is its service for renting servers.
Instance	One rented server.
Elastic IP	A fixed internet address for your server.
Security group	The server's firewall rules, set in the AWS console.
SSH	A secure way to type commands on the server from your computer.
Key pair / .pem / deploy key	Files that prove who you are. Keep them private.
Terminal	The window where you type commands.
Docker / container	A packaged, isolated copy of an app. Each part of the platform is a container.
docker compose	The tool that starts several containers together, using <code>docker-compose.prod.yml</code> .
Environment file (.env)	A text file of settings and secrets read by the containers.
Cloudflare Tunnel	The secure link between Cloudflare and your server.
HTTPS / padlock	Encrypted website traffic, provided by Cloudflare here.
Migration	A script that changes the database structure (step 25).
Seed	Loading sample data into the database (step 26).
Swap	Disk space used as extra memory.